

Tesorería

Módulo de Administración de Layouts Bancarios

Cobranza Domiciliada



Arquitectura: React + ASP.NET Core Web API + Entity Framework Core + MySQL

Solano Jasso Diego Samuel

Fecha: 4 de septiembre de 2026

Versión 1.0

Cobranza Domiciliada (Área de Finanzas)

RF2	PANTALLA PRINCIPAL PARA LAYOUTS BANCARIOS Dentro del submenú “Cobranza Domiciliada” se deberá de contar con una pantalla denominada “Cobranza Domiciliada” Dicha pantalla requiere contar con un botón (+ Nuevo) para dar de alta nuevos layouts bancarios. Esta funcionalidad se detalla en siguiente funcional: Frecuencia posible de cambio de layouts nuevos: • Cada 6 meses revisión con instituciones bancarias Frecuencia posible de cambio de layouts existentes en la estructura: • Cada 5 años Frecuencia posible de cambio de layouts existentes en datos: • Cada 2 años Se requiere contar con un filtro de búsqueda con los siguientes campos para captura del usuario: • Nombre del usuario • Banco • Estatus Botón buscar: Accionara la búsqueda de acuerdo con los filtros integrados Botón información de apoyo layout cobranza El resultado de la búsqueda deberá de contar con las siguientes columnas: • Nombre del layout • Banco • Estatus • Acciones (Ver detalle, copiar y editar)
-----	---

1. Propósito principal u objetivo general:

Facilitar la búsqueda de layouts bancarios para los usuarios que accedan al submenú de Cobranza Domiciliada.

Permitirá buscar de manera rápida información o estatus de la información a consultar.

2. ¿Qué va a solucionar?

Este requisito funcional nos permitirá dar solución a la problemática donde los usuarios que acceden a la subpágina de Layouts bancarios, puedan encontrar la información que ellos buscan de manera rápida y ordenada de la forma en la que ellos necesiten, ya sea, buscando por medio del nombre del layout, banco del que se recibe o envía el layout o por medio de su estatus actual en el sistema.

3. Objetivo Especifico:

Se está solicitando que se cree una pagina dentro del menú “Cobranza Domiciliada” que les permita a los usuarios que trabajen en ella poder visualizar la información almacenada. La información que se tiene que visualizar es:

1. Nombre del layout
2. Banco
3. Estatus actual del layout
4. Y las acciones que se pueden realizar relacionadas a ese layout (Ver detalle, Copia y Editar)

Además, se podrán hacer búsquedas ya sea por medio del Layout Bancario, por nombre del Banco o Estatus actual de los layouts almacenados.

Aun que esos layouts se desactiven, no serán borrados de la memoria para poder seguir visualizando y teniendo un historial de modificaciones.

1. Resumen ejecutivo

La presente propuesta técnica documenta la solución desarrollada para la administración de layouts bancarios utilizados en el proceso de Cobranza Domiciliada. El módulo permite consultar, filtrar, crear, visualizar, editar y copiar layouts, manteniendo una separación clara entre la interfaz web, la API y la base de datos.

La solución se implementó con React y Vite en el frontend, ASP.NET Core Web API sobre .NET 8 en el backend, Entity Framework Core como capa de acceso a datos y MySQL como motor de persistencia. La base de datos se ejecuta en un contenedor Docker para facilitar la configuración y reproducibilidad del entorno.

Estado actual

El alcance descrito en este documento corresponde a funcionalidades ya implementadas y probadas en el módulo. Las mejoras de seguridad, auditoría, paginación y autenticación se consideran extensiones futuras y no se presentan como funciones ya terminadas.

2. Objetivo

Desarrollar un módulo web que permita administrar los layouts asociados a las instituciones bancarias utilizadas dentro del proceso de cobranza domiciliada, con una experiencia de uso clara y con controles básicos para disminuir errores operativos.

- Consultar los layouts bancarios registrados.
- Buscar layouts por nombre y filtrar por banco y estatus.
- Registrar nuevos layouts bancarios.
- Visualizar el detalle de un layout.
- Modificar el nombre, banco y estatus de un layout existente.
- Crear una copia de un layout mediante confirmación previa.
- Mostrar notificaciones de éxito y error al usuario.
- Evitar operaciones duplicadas durante acciones en procesamiento.

3. Alcance funcional

El alcance actual se concentra en el catálogo de Layouts Bancarios y su relación con el catálogo de Bancos. Cada layout contiene un identificador único, nombre, banco asociado y estatus activo/inactivo.

Campo	Descripción
Id	Identificador único del layout.
Nombre	Nombre descriptivo del layout bancario.
Banco	Institución bancaria asociada.
Estatus	Indica si el layout se encuentra activo o inactivo.

4. Arquitectura de la solución

La aplicación se divide en tres componentes principales: presentación, servicios y persistencia. Esta separación permite mantener responsabilidades claramente definidas y facilita la incorporación de cambios futuros.



5. Tecnologías utilizadas

Capa	Tecnologías	Propósito
Frontend	React, JavaScript, Vite, HTML, CSS, Fetch API	Interfaz de usuario, formularios, modales, filtros y consumo de la API.
Backend	ASP.NET Core Web API, .NET 8, C#	Exposición de endpoints, validaciones y lógica de aplicación.
Persistencia	Entity Framework Core, Pomelo MySQL	Mapeo de entidades y acceso a MySQL.
Base de datos	MySQL	Almacenamiento persistente de bancos y layouts.
Entorno	Docker	Ejecución aislada y reproducible de MySQL.

6. Modelo de datos

La base de datos contiene las entidades Bancos y Layouts. Cada layout almacena una llave foránea Bancold que referencia al banco al que pertenece.

Tabla	Campos principales	Descripción
Bancos	Id, Nombre	Catálogo de instituciones bancarias.
Layouts	Id, Name, Estado, Bancold	Configuración general de layouts bancarios.

Relación

Bancos (1) ————— (N) Layouts. Un banco puede estar asociado a múltiples layouts; cada layout pertenece a un solo banco.

7. Diseño de la API REST

La API expone endpoints bajo el prefijo /Tesorería. Los endpoint implementados permiten consultar, crear, modificar y copiar layouts, además de obtener el catálogo de bancos.

Método	Endpoint	Función
GET	/Tesoreria/LayoutBancario	Obtiene layouts y acepta filtros opcionales.
GET	/Tesoreria/LayoutBancario/{id}	Obtiene el detalle de un layout.
POST	/Tesoreria/LayoutBancario	Crea un nuevo layout.
PUT	/Tesoreria/LayoutBancario/{id}	Actualiza un layout existente.
POST	/Tesoreria/LayoutBancario/{id}/copiar	Genera una copia de un layout.
GET	/Tesoreria/Banco	Obtiene el catálogo de bancos.

7.1 Filtros de consulta

El endpoint de listado admite parámetros opcionales para construir búsquedas sin requerir rutas adicionales:

- nombre: filtra por coincidencia en el nombre del layout.
- bancold: filtra por identificador de banco.
- estatus: filtra por estado activo o inactivo.

Ejemplo

```
GET /Tesoreria/LayoutBancario?nombre=BBVA&estatus=true
```

7.2 Validaciones del backend

- Comprobación de existencia del layout antes de consultar, modificar o copiar.
- Comprobación de existencia del banco antes de crear o actualizar.
- Uso de identificadores Guid para layouts y bancos.
- Respuestas HTTP acordes al resultado de la operación.

Código	Uso
200 OK	Consulta o actualización realizada correctamente.
201 Created	Nuevo layout creado correctamente.
400 Bad Request	Solicitud no válida, por ejemplo un banco inexistente.
404 Not Found	No se encontró el layout solicitado.

8. Interfaz de usuario

La pantalla principal presenta los layouts en una tabla con las columnas Nombre, Banco, Estatus y Acciones. La interfaz incluye controles de búsqueda, modales para operaciones y notificaciones temporales.

Elemento	Comportamiento
Buscar	Aplica filtros de nombre, banco y estatus.
Limpiar	Restablece filtros y recupera el listado completo.
Crear Layout	Abre un modal para registrar un nuevo layout.
Ver	Consulta el detalle y lo presenta en un modal.
Editar	Carga la información actual y permite modificarla.
Copiar	Abre una confirmación antes de crear una copia.

8.1 Creación de layouts

El modal de creación solicita nombre, banco y estatus. Antes de enviar la solicitud se valida que el nombre no esté vacío y que se haya seleccionado un banco. Cuando la operación termina correctamente, el modal se cierra, la tabla se actualiza y se muestra una notificación de éxito.

8.2 Consulta de detalle

La acción Ver consulta el layout seleccionado en la API y muestra nombre, banco y estatus dentro de una ventana modal, evitando que el usuario abandone la pantalla principal.

8.3 Edición

La acción Editar obtiene la información actual del layout y precarga un formulario. El usuario puede modificar nombre, banco y estatus. Tras una actualización exitosa, la tabla se vuelve a consultar y se informa el resultado mediante una notificación.

8.4 Confirmación de copia

La acción Copiar no genera un registro inmediatamente. Primero se muestra un modal de confirmación con el nombre y banco del layout seleccionado. El usuario puede cancelar o confirmar la operación. Solo después de confirmar se ejecuta el POST de copia.

9. Control de operaciones y experiencia de usuario

La interfaz incorpora un estado de procesamiento para evitar que una acción sensible se ejecute varias veces por doble clic. Durante la copia, el botón cambia de “Copiar” a “Copiando...” y se deshabilitan los controles relacionados hasta que la solicitud finaliza.

- Bloqueo temporal del botón durante el procesamiento.
- Protección lógica para evitar ejecutar la misma función nuevamente mientras existe una operación activa.
- Restablecimiento del estado tanto en éxito como en error.
- Mensajes visuales para comunicar el resultado de las operaciones.

10. Sistema de notificaciones

El frontend muestra mensajes temporales en la parte superior derecha para informar al usuario sobre operaciones completadas o errores. Las notificaciones desaparecen automáticamente después de unos segundos.

Tipo	Ejemplo
Éxito	“Layout creado. El layout se creó correctamente.”
Éxito	“Layout actualizado. Los cambios se guardaron correctamente.”
Éxito	“Layout copiado. El layout se copió correctamente.”
Error	“No se pudo actualizar el layout.”
Validación	“Debes ingresar un nombre para el layout.”

11. Comunicación Frontend - Backend

El frontend consume la API mediante Fetch API. Las solicitudes y respuestas se intercambian en formato JSON. Después de una operación de creación, edición o copia, la interfaz vuelve a consultar el endpoint de listado para reflejar el estado actual de la base de datos.

Flujo general

Usuario → React → HTTP/JSON → ASP.NET Core API → Entity Framework Core → MySQL → Respuesta → Actualización de interfaz

12. Configuración CORS

Durante el desarrollo, el frontend y la API se ejecutan en orígenes diferentes. La API permite solicitudes desde el origen del frontend mediante una política CORS específica.

Componente	Dirección de desarrollo
Frontend	http://localhost:5173
API	http://localhost:5020

13. Beneficios de la solución

- Centraliza la administración de layouts bancarios.
- Separa la presentación, lógica de aplicación y persistencia.
- Permite búsquedas y filtros sin navegar a otras pantallas.
- Reutiliza configuraciones mediante la función de copia.
- Reduce errores accidentales mediante confirmaciones y bloqueo de acciones.
- Mantiene el catálogo de bancos desacoplado de la interfaz.
- Facilita la incorporación de nuevos requisitos sin rehacer la solución completa.

14. Consideraciones de seguridad

La versión actual incorpora validaciones de integridad relacionadas con layouts y bancos. Para un entorno productivo se recomienda ampliar la solución con controles de seguridad adicionales.

- Autenticación de usuarios.
- Autorización basada en roles y permisos.
- Registro de auditoría de operaciones.
- Manejo centralizado de excepciones.
- Administración segura de cadenas de conexión y secretos.
- Uso obligatorio de HTTPS en ambientes publicados.
- Validaciones adicionales del lado servidor.

15. Mejoras futuras

La arquitectura actual permite incorporar nuevas funciones de forma incremental conforme se definan nuevos requisitos.

- Baja lógica o eliminación controlada de layouts.
- Historial de modificaciones y auditoría por usuario.
- Paginación y ordenamiento de resultados.
- Autenticación y autorización.
- Exportación de información.
- Administración del contenido interno de cada layout.
- Integración con otros procesos de cobranza.
- Manejo global de errores.
- Variables de entorno para las direcciones y configuraciones de servicios.

16. Conclusión

La solución implementada proporciona una base funcional para la administración de layouts bancarios asociados al proceso de Cobranza Domiciliada. El módulo permite actualmente consultar, filtrar, crear,

visualizar, editar y copiar layouts, e incorpora validaciones, modales, confirmaciones, notificaciones y controles para prevenir operaciones duplicadas.

El uso de React, ASP.NET Core, Entity Framework Core y MySQL mantiene una arquitectura clara y extensible, permitiendo que el sistema continúe evolucionando conforme se incorporen nuevos requisitos funcionales, operativos o de seguridad.

Anexo A. Resumen de funcionalidades implementadas

Funcionalidad	Estado
Listado de layouts	Implementado
Filtro por nombre	Implementado
Filtro por banco	Implementado
Filtro por estatus	Implementado
Crear layout	Implementado
Ver detalle	Implementado
Editar layout	Implementado
Copiar layout	Implementado
Confirmación antes de copiar	Implementado
Notificaciones visuales	Implementado
Bloqueo durante procesamiento de copia	Implementado
Autenticación / autorización	Mejora futura
Auditoría	Mejora futura